SUNWAY UNIVERSITY

CSC3209 - Software Architecture and Design Pattern – Practical 1

1- What is software architecture? And What makes an architecture good?

Answer: The software architecture of a system is the set of structures needed to reason about the system which comprises software elements relations among them, and properties of both.

Several aspects make the architecture a good architecture.

- Architectures are either more or less should fit for some purpose.
- The architecture should be the product of a single architect or a small group of architects with an identified technical leader.
- The architect or architecture team) should base the architecture on a prioritized list of well specified quality attribute requirements.
- The architecture should be documented using views. The views should address the concerns of the most important stakeholders in support of the project timeline.
- The architecture should be evaluated for its ability to deliver the system's important quality attributes.
- The architecture should lend itself to incremental implementation.
- The architecture should feature well defined modules whose functional responsibilities are assigned on the principles of information hiding and separation of concerns.
- The architecture should never depend on a particular version of a commercial product or tool.

2- What is the difference between an architectural structure and view?

Answer: A structure is the set of elements itself as they exist in software or hardware. A view is a representation of a coherent set of architectural elements as written by and read by system stakeholders. In short, a view is a representation of a structure.

3- There are three categories of architectural structures:

- Module structures
- Component-and-connector structures
- Allocation structures

For those structures:

a) Explain how the structure is used and what concerns are addressed in the structure.

Answer:

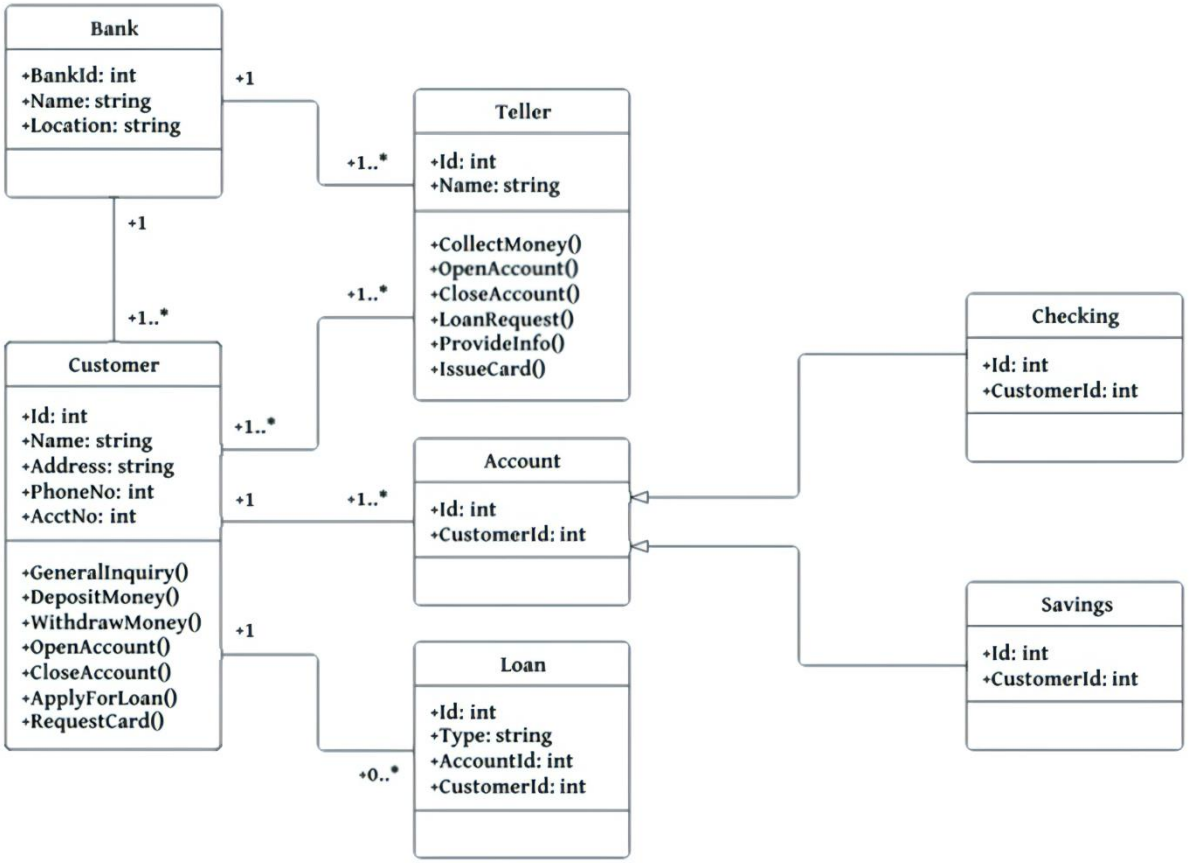
Module structures Embody decisions as to how the system is to be structured as a set of code or data units that have to be constructed or procured. In any module structure, the elements are modules of some kind (perhaps classes, or layers, or merely divisions of functionality, all of which are units of implementation). Modules are assigned areas of functional responsibility; there is less emphasis in these structures on how the software manifests at runtime. Module structures allow us to answer questions such as these: What is the primary functional responsibility assigned to each module? What other software elements is a module allowed to use? What other software does it actually use and depend on? What modules are related to other modules by generalization or specialization (i.e., inheritance)?

Component and connector structures Embody decisions as to how the system is to be structured as a set of elements. Elements are runtime components such as services, peers, clients, servers, filters, or many other types of runtime element) that have runtime behavior (components) and interactions (connectors). Connectors are the communication vehicles among component, such as call return, process synchronization operators, pipes, or others. Component and connector views help us answer questions such as these: What are the major executing components and how do they interact at runtime? What are the major shared data stores? Which parts of the system are replicated? How does data progress through the system? What parts of the system can run in parallel? Can the system's structure change as it executes and, if so, how?

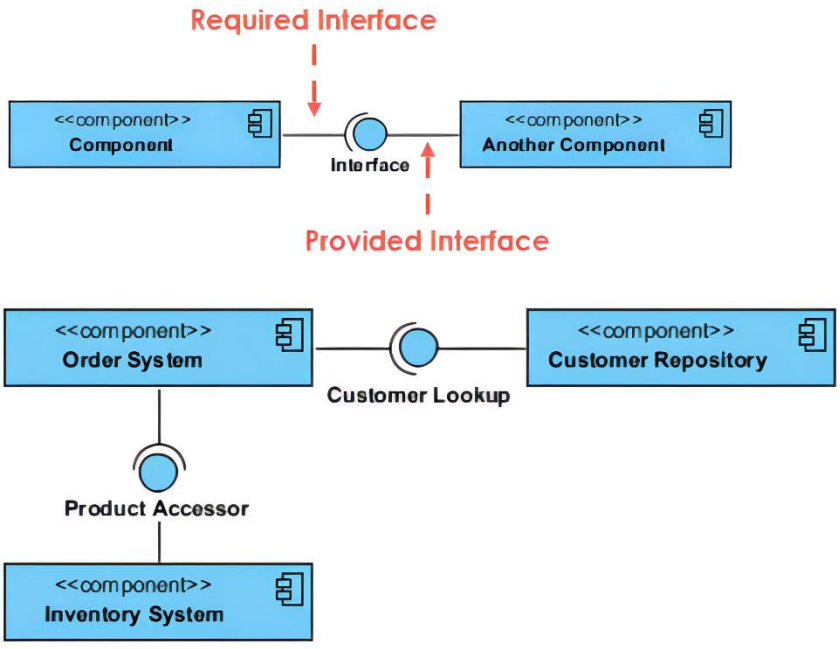
Allocation structures Show the relationship between the software elements and elements in one or more external environments in which the software is created and executed. Allocation views help us answer questions such as these: What processor does each software element execute on? In what directories or files is each element stored during development, testing, and system building? What is the assignment of each software element to development teams?

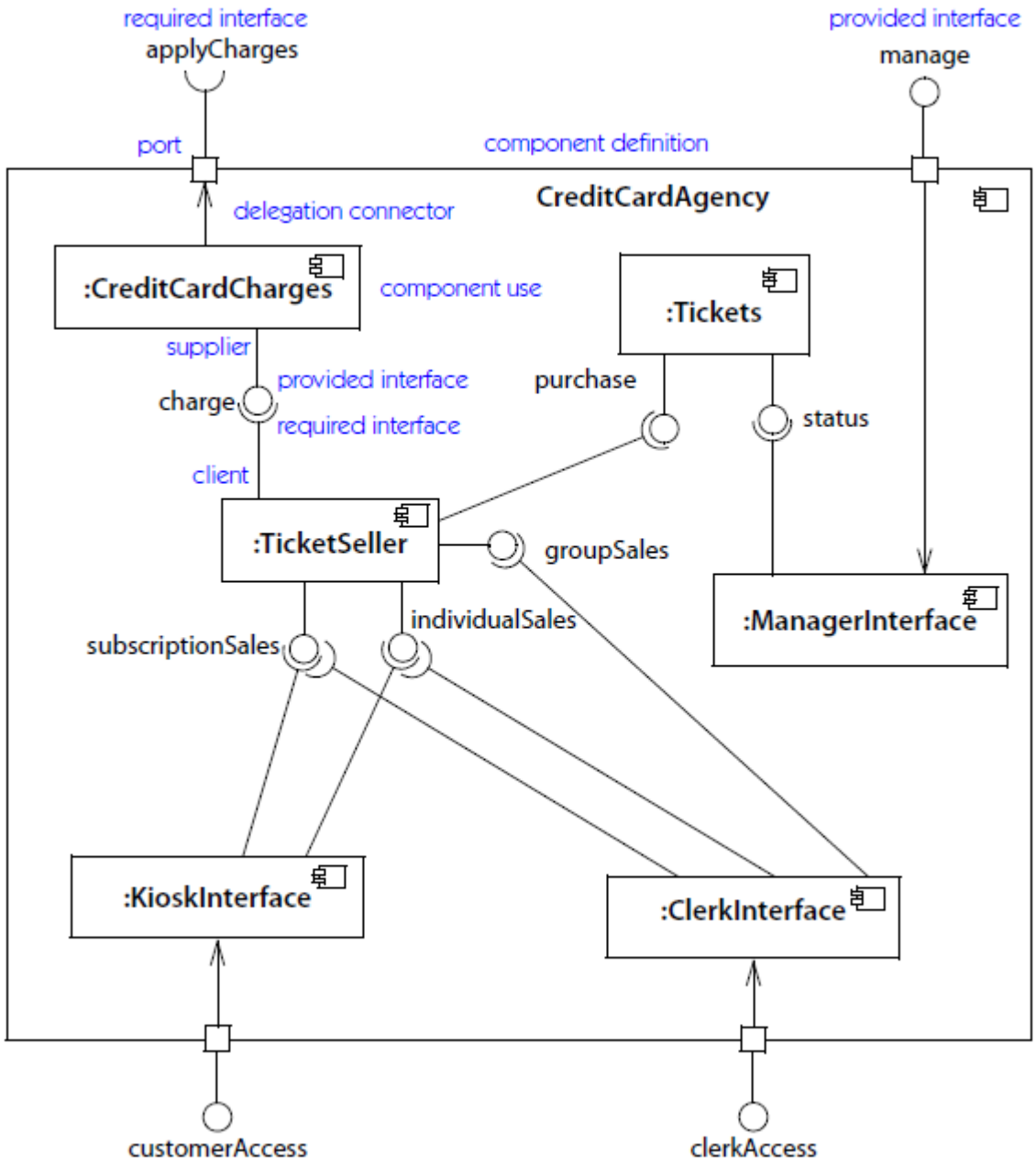
b) Give some examples of every type of structure.

Answer: **Module structure: Class (or generalization) structure**: UML Class Diagram. The module units in this structure are called classes. Some relations are "inherits from" or "is an instance of". This view supports reasoning about collections of similar behavior or capability.

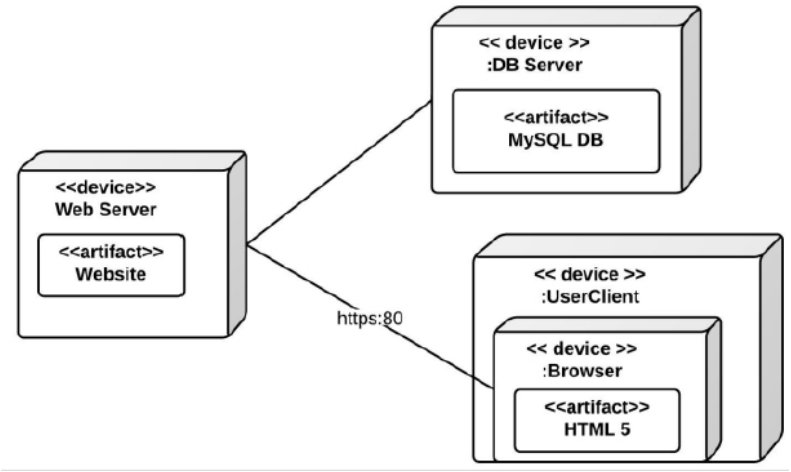


Component-and-Connect structure: UML component diagrams.





Allocation Structures: Deployment structure: It shows how software is assigned to hardware processing and communication elements. The elements are software elements (usually a process from a C&C view), hardware entities (and communication pathways. Relations are allocated to showing on which physical units the software elements reside and migrates to if the allocation is dynamic. UML deployment diagram is an example of allocation structures and deployment structure.



c) Discuss the concerns that are addressed in every example.

The module structure (class structure) allows one to reason about reuse and the incremental addition of functionality. Class diagram allows to reason about: every functionality provided by every class, how every class is allowed (via i.e. association) to use other classes, what classes are related to other classes via generalization.

The component-and-connector structure (the component diagram) is useful in construction for specifying the behavior that elements must exhibit, analysis for reasoning about runtime system quality attributes such as performance, security, and reliability.

The allocation (Deployment structure) could be used to reason about: what hardware and software is needed, Distributed development and allocation of work to teams, Builds, integration testing, version control and system installation.